I am a student developing a real implementation of:

Problem Statement ID: 26080
Title: Regime-Aware AI Post-Processing of Monsoon Rainfall Forecasts
Organization: Ministry of Earth Sciences (MoES)
Department: National Centre for Medium Range Weather Forecasting (NCMRWF)
The objective is NOT to create a theoretical proposal or a fake/demo AI system.
I want to build a real, reproducible, student-level implementation that:
obtains real historical weather forecast data,
obtains real observed rainfall data,
correctly aligns forecast and observation,
establishes a raw-NWP baseline,
improves the rainfall prediction using ML,
adds weather-regime information,
measures whether the improvement is real,
produces a usable district/grid-level rainfall product.
I need a research document that can serve as our technical development specification.
CRITICAL INSTRUCTION — DO NOT HALLUCINATE
This is extremely important.
Do NOT invent:
datasets
download URLs
API endpoints
variables
forecast fields
historical archive availability
model resolutions
forecast accumulation definitions
regime labels
IMD thresholds
NCMRWF capabilities
published research results
performance numbers
For every important factual statement, provide a source.
Prefer official sources:
NCMRWF
Ministry of Earth Sciences
IMD
NOAA/NCEP/NCEI
ECMWF

WMO
peer-reviewed papers
official dataset documentation
If something cannot be verified, explicitly write:
"Not verified — do not implement this yet."
If a dataset is theoretically available but difficult/inaccessible for students, say so.
Do not silently replace one dataset with another.
PART 1 — FREEZE THE ACTUAL PROJECT SCOPE
First translate PS 26080 into a technically achievable student project.
The official requirements are:
A. Weather regime classifier
Classify regimes such as:
active monsoon
break monsoon
depression/low
coastal rainfall
orographic rainfall
B. Bias-corrected rainfall forecast
Improve raw NWP rainfall.
C. Heavy rainfall probability
Estimate probability that rainfall exceeds an operational threshold.
D. District/grid rainfall product
Display predicted rainfall on a map/table.
E. Verification
Use:
RMSE
ETS
CSI
POD
FAR
FSS
Determine which of these can realistically be implemented.
Then define the smallest complete version that satisfies the problem statement without pretending to be an operational meteorological system.
PART 2 — CHOOSE THE ACTUAL DATA SOURCES
Investigate these candidates separately.
A. NCMRWF / NCUM
Determine:
What raw rainfall forecast data NCMRWF produces
Whether historical raw NCUM forecast data is publicly downloadable
Exact access mechanism
Historical period available
Resolution
Forecast cycles
Forecast lead times
Variables

File format
IMPORTANT:
If historical raw NCUM data cannot realistically be obtained by a student, clearly say:
"NCUM will be treated as the intended operational target, but not used as the experimental dataset."
Do not pretend GFS is NCUM.
PART 3 — GFS AS A POSSIBLE REPRODUCIBLE NWP SOURCE
Investigate current and historical NOAA GFS data.
Verify from official NOAA/NCEP/NCEI documentation:
exact GFS product
model version/cycle
historical archive period
spatial resolution
forecast cycles
forecast hours
temporal resolution
precipitation variables
units
GRIB2 structure
access method
whether historical files can actually be downloaded today
Compare:
GFS 0.25°
GFS 0.5°
Determine which is more practical for this student project.
Do not choose based only on resolution.
Consider:
historical availability
storage
download volume
processing
model variables
compatibility with IMD data
reproducibility
PART 4 — ACTUAL DOWNLOAD TEST SPECIFICATION
This is critical.
Find the exact official URL/path/API pattern for downloading ONE historical GFS forecast.
Use a concrete example date that is actually available.
For example:
2023-07-01 00 UTC
But verify that this exact data exists before using it.
Give:
exact URL/path
exact file name
forecast cycle
forecast hour

file format
expected variables
approximate file size
If an exact historical URL cannot be verified, do NOT fabricate one.
Instead provide the verified archive navigation method.
PART 5 — PRECIPITATION FIELD
Determine exactly which GFS field should represent forecast rainfall.
Answer:
variable name
GRIB parameter
units
accumulation period
forecast-hour meaning
whether it is instantaneous or accumulated
how accumulated rainfall changes between forecast steps
how to calculate a daily/24-hour rainfall amount
Give an exact worked example.
Example format:
Initialization:
YYYY-MM-DD 00 UTC

Forecast hour:
...

Field:
...

Accumulation:
...

Target rainfall period:
...

Observed rainfall period:
...

Do NOT assume that "APCP at FH24" automatically means the exact 24-hour period we need.
Verify the accumulation semantics from official documentation.
PART 6 — IMD OBSERVED RAINFALL
Use the official IMD 0.25° daily gridded rainfall dataset if feasible.
Verify:
current period available
spatial grid
latitude/longitude definition
temporal definition
units
file format
missing values

download method
official access
sample file
whether 2023/2024 data is available
Give the exact official source.
Determine whether NetCDF is available.
Prefer NetCDF if officially available because it is easier to process reproducibly in Python.
PART 7 — FORECAST + OBSERVATION ALIGNMENT
Design the exact matching algorithm.
We need:
Forecast issue time
+
forecast lead time
+
forecast accumulation
observation period
Explain:
UTC handling
date boundaries
leap years
spatial coordinates
longitude conventions
grid orientation
missing values
Create a concrete example.
Then define:
forecast_date
initialization_time
lead_time
valid_start
valid_end
latitude
longitude
nwp_rainfall
observed_rainfall

PART 8 — SPATIAL MATCHING
Determine how to align:
GFS grid
with:
IMD 0.25° grid
Compare:
Option 1
Interpolate GFS → IMD grid
Option 2
Aggregate IMD → GFS grid

Option 3
Use common grid/domain
Research which approach is scientifically defensible for rainfall.
Pay particular attention to extreme rainfall.
Recommend ONE method for our first experiment.
Explain why.
PART 9 — DEFINE THE FIRST EXPERIMENT
Do not start nationwide with every regime.
Design a minimum viable but scientifically valid experiment.
Investigate whether the following is practical:
Geographic domain
India or monsoon core region
Season
June–September
Forecast
24-hour rainfall
NWP
GFS
Observation
IMD 0.25°
Initial period
Choose years based on actual data availability.
Explain why.
PART 10 — DATASET SIZE
Calculate approximate data volume.
For example:
number of days
number of grid cells
number of samples
number of variables
expected CSV/Parquet size
GRIB storage
processed dataset storage
Recommend:
Prototype dataset
Development dataset
Final experiment dataset
Do not recommend 10+ years if it creates unnecessary storage/processing requirements.
PART 11 — EXACT TRAINING TABLE
Design the final ML dataset.
Example:
forecast_init
valid_date
lead_time
latitude
longitude

nwp_rain
temperature
relative_humidity
wind_u
wind_v
pressure
geopotential
cape
observed_rain
regime

But DO NOT assume all these variables exist.
For every proposed feature mark:
VERIFIED AVAILABLE
OPTIONAL
NOT VERIFIED
Then produce the final minimal feature set.
PART 12 — BASELINE 1: RAW NWP
Define the first experiment.
Input:
GFS rainfall
Target:
IMD rainfall
No ML.
Calculate:
RMSE
MAE
bias
CSI
POD
FAR
ETS
FSS
for appropriate rainfall thresholds.
This establishes:
How good is the raw NWP forecast?
Provide Python implementation guidance.
Do NOT provide code yet unless it is required to clarify the procedure.
PART 13 — BASELINE 2: STATISTICAL BIAS CORRECTION
Compare:
mean bias correction
linear scaling
quantile mapping
Determine which is easiest and scientifically defensible.
Then choose ONE primary baseline.
Define:

Training procedure
Validation procedure
Test procedure
Formula
Required inputs
Output
Make sure correction parameters are calculated ONLY from training data.
No test-set leakage.
PART 14 — BASELINE 3: GENERIC ML
Compare:
Random Forest
XGBoost
LightGBM
For this problem.
Consider:
tabular data
nonlinear relationships
rainfall skewness
zero rainfall
heavy rainfall
CPU training
explainability
student implementation complexity
Choose ONE primary model.
Most likely XGBoost, but do not automatically choose it. Justify the choice using evidence.
PART 15 — RAINFALL ZERO-INFLATION
Research this carefully.
Rainfall contains many zero values.
Determine whether our ML model should use:
Approach A
Single regression model
Approach B
Two-stage model:
Rain / No Rain classifier
↓
Rainfall amount regression

Approach C
Other scientifically supported approach
Determine the simplest method that improves rainfall post-processing without making the project unnecessarily complex.
PART 16 — WEATHER REGIME CLASSIFICATION
This is the core innovation.
But do NOT invent regime labels.
Investigate actual scientific/operational definitions for:
Active monsoon

Break monsoon
Depression/low
For each identify:
source
definition
required variables
spatial/temporal criteria
availability of historical labels
Determine whether we can realistically create labels.
PART 17 — START WITH FEWER REGIMES
Determine whether the first implementation should use:
Active
Break
Depression/Low
Other/Normal

instead of six separate regimes.
Evaluate whether this satisfies the core idea of regime-aware post-processing.
If not, explain what minimum regime set is required.
PART 18 — REGIME CLASSIFIER
Compare realistic methods:
rule-based classification
logistic regression
random forest
XGBoost
clustering
SOM
Do NOT recommend a complex classifier unless necessary.
Determine whether a rule-based classifier using published scientific criteria could actually be more defensible than ML for the first version.
PART 19 — REGIME-AWARE MODEL
Compare:
Architecture A
Separate model per regime:
Active → Model A
Break → Model B
Depression → Model C

Architecture B
Single model:
NWP variables + regime
↓
XGBoost

Architecture C
Mixture of experts
Determine which architecture:

is trainable with limited data
avoids sparse-regime problems
is easy to explain
is computationally feasible
directly tests the PS 26080 hypothesis
Choose ONE.
PART 20 — HEAVY RAINFALL PROBABILITY
Research official IMD thresholds.
Then determine a practical implementation.
Possible:
P(Rain > threshold)

Investigate whether we should use:
binary classifier
quantile regression
ensemble predictions
calibrated probability
other approach
Choose the simplest scientifically valid option.
Explain how probability should be evaluated.
PART 21 — FINAL MODEL COMPARISON
Our final experiment should compare:
B1 Raw NWP
↓
B2 Bias Correction
↓
B3 Generic ML
↓
B4 Regime-Aware ML

Create the exact experiment matrix.
For each model define:
input
target
training period
validation period
test period
metrics
expected output
DO NOT provide expected numerical results.
We will calculate them ourselves.
PART 22 — PROVE IMPROVEMENT
Define exactly how we can legitimately claim:
"Our model improved the rainfall forecast."
For example:
lower RMSE
higher CSI

higher POD
lower FAR
higher ETS
better FSS
But explain which metrics must improve together.
Also explain why one metric alone is insufficient.
PART 23 — EXTREME RAINFALL EVALUATION
Evaluate separately for:
no/low rainfall
moderate rainfall
heavy rainfall
very heavy rainfall
extreme rainfall if data supports it
Determine how many events are needed before conclusions are meaningful.
Avoid misleading results caused by class imbalance.
PART 24 — SPATIAL EVALUATION
Determine how to calculate FSS.
Explain:
neighborhood size
rainfall threshold
spatial forecast grid
observed grid
why FSS helps with spatial displacement
Recommend practical neighborhood sizes based on grid resolution.
PART 25 — DISTRICT-LEVEL PRODUCT
Only after the scientific model works, determine how to produce district forecasts.
Investigate:
official India district boundaries
source
GeoJSON/shapefile
historical/current boundary problems
Determine whether we should:
Option A
Predict directly on grid and aggregate to district
or
Option B
Train a district-specific model
Recommend one.
Prefer the approach that preserves the meteorological grid information.
PART 26 — SOFTWARE ARCHITECTURE
Design a simple architecture.
Prefer:
Python
├── data/
├── preprocessing/
├── regime/

├── models/
├── evaluation/
├── visualization/
└── app/

Recommend appropriate libraries ONLY where needed.
Potential libraries:
xarray
cfgrib
eccodes
pandas
numpy
scikit-learn
xgboost
matplotlib
geopandas
rasterio
netCDF4
cartopy
FastAPI
Streamlit
Do not add libraries unnecessarily.
Explain exactly why each library is needed.
PART 27 — EXACT DEVELOPMENT PIPELINE
Create the complete pipeline:
01_download_gfs
↓
02_download_imd
↓
03_read_raw_data
↓
04_validate_metadata
↓
05_align_time
↓
06_align_grid
↓
07_create_training_dataset
↓
08_create_regime_labels
↓
09_train_baseline
↓
10_train_bias_correction
↓
11_train_xgboost

↓
12_train_regime_aware_model
↓
13_evaluate
↓
14_generate_maps
↓
15_generate_district_product
↓
16_dashboard

For each stage specify:
input
output
file format
validation checks
failure conditions
PART 28 — PROJECT FOLDER STRUCTURE
Create an exact practical folder structure.
For example:

```
PS26080/
│
├── data/
│   ├── raw/
│   │   ├── gfs/
│   │   └── imd/
│   ├── interim/
│   └── processed/
│
├── notebooks/
│
├── src/
│   ├── data/
│   ├── preprocessing/
│   ├── regimes/
│   ├── models/
│   ├── evaluation/
│   └── visualization/
│
├── configs/
├── models/
├── outputs/
├── reports/
├── tests/
├── requirements.txt
└── README.md
```

Modify this according to the actual implementation.
PART 29 — CODE EXECUTION PLAN
This is very important.
Create a sequence of coding milestones.
Milestone 0
Verify data access
Milestone 1
Download one forecast
Milestone 2
Read GRIB2
Milestone 3
Read IMD NetCDF
Milestone 4
Match one day
Milestone 5
Create 7-day sample
Milestone 6
Create one-month dataset
Milestone 7
Run raw-NWP verification
Milestone 8
Bias correction
Milestone 9
XGBoost
Milestone 10
Regime labels
Milestone 11
Regime-aware XGBoost
Milestone 12
Heavy rainfall probability
Milestone 13
District aggregation
Milestone 14
Dashboard
For each milestone specify:
What should work before moving to the next milestone.
PART 30 — STOP CONDITIONS
Define conditions where we should STOP and reassess.
Examples:
Stop condition 1
Historical GFS forecast cannot be accessed.
Stop condition 2
GFS rainfall accumulation cannot be reliably interpreted.
Stop condition 3
IMD observations cannot be matched.
Stop condition 4

Regime labels cannot be created reliably.
Stop condition 5
No improvement over raw NWP.
Stop condition 6
Model improves RMSE but worsens heavy-rain detection.
For each condition provide a scientifically honest fallback.
PART 31 — FALLBACK STRATEGY
Create a hierarchy:
Level 1 — Ideal
NCMRWF NCUM + IMD
Level 2 — Reproducible
GFS + IMD
Level 3 — Reduced prototype
Small India domain + GFS + IMD
Level 4 — Algorithm validation only
Small sample / synthetic test
Clearly explain that Level 4 is NOT sufficient to claim real forecasting improvement.
PART 32 — WHAT NOT TO BUILD
Explicitly tell us what NOT to implement initially.
Examples:
custom NWP model
huge deep-learning architecture
Transformer unless justified
real-time operational deployment
10-day forecasting
all six regimes immediately
complex microservices
unnecessary mobile application
unnecessary IoT hardware
fabricated live forecast capability
Focus on scientific validation first.
PART 33 — FINAL TECHNICAL SPECIFICATION
At the end produce one consolidated specification titled:
"PS 26080 — Student Implementation Technical Specification"
It must contain:

1. Problem definition
2. Data sources
3. Verified data access
4. Exact forecast variable
5. Exact observation variable
6. Temporal matching
7. Spatial matching
8. Training dataset schema

9. Train/validation/test split
10. Baseline B1
11. Bias correction B2
12. Generic ML B3
13. Regime classifier
14. Regime-aware ML B4
15. Heavy rainfall probability
16. Evaluation metrics
17. Spatial evaluation
18. District product
19. Software architecture
20. Folder structure
21. Python libraries
22. Development milestones
23. Validation checks
24. Stop conditions
25. Fallback plan
26. Known limitations
27. Claims we are allowed to make
28. Claims we must NOT make

    FINAL RULE

    The final plan must be executable by a student team using ordinary laptops or modest cloud/CPU resources wherever possible.

    Do not design a system that requires:

    expensive GPU clusters
    proprietary data
    inaccessible NCMRWF archives
    undocumented APIs
    massive datasets
    unrealistic training time

    If a scientifically desirable component is not practically accessible, explicitly mark it as:
    FUTURE / OPTIONAL — NOT REQUIRED FOR THE CORE EXPERIMENT.

    The core project must remain:

    Real forecast data → real observations → correct alignment → raw baseline → ML correction → regime-aware correction → objective verification.

    Do not write the Python implementation yet.

    First give me the verified technical execution plan that another student should be able to follow without guessing.

Based on my extensive research, I can now provide you with a comprehensive, verified technical specification. Where information cannot be verified from official sources, I have explicitly marked it as "NOT VERIFIED — DO NOT IMPLEMENT YET."

PS 26080 — Student Implementation Technical Specification

Regime-Aware AI Post-Processing of Monsoon Rainfall Forecasts

Version: 1.0
Date: 26 September 2026
Target: Student-level reproducible implementation (ordinary laptops, modest cloud/CPU resources)

1. Problem Definition (Translated from PS 26080)

Objective: Build a real, reproducible ML system that improves 24-hour rainfall forecasts over India during the monsoon season by:

1. Classifying weather regimes (active, break, depression/low)
2. Applying bias correction to raw NWP rainfall
3. Estimating heavy rainfall probability
4. Producing district/grid-level rainfall products
5. Verifying improvement using standard meteorological metrics

Scope constraint: This is **not** an operational forecasting system. It is a scientifically valid prototype using publicly accessible data.

2. Data Sources

2.1 Forecast Data (NWP)

Property	Value	Source
Model	NOAA Global Forecast System (GFS)	[1] [2]
Resolution	0.25° (preferred for this project)	[1] [2]
Forecast cycles	00, 06, 12, 18 UTC (4/day)	[1]
Forecast hours	FH000, FH003–FH384 (3-hourly output)	[2]
File format	GRIB2 (pgrb2.0p25)	[3] [2]
Historical archive	26 Feb 2021 – Present (0.25°)	[1]
Access	AWS S3 (no account required)	[4] [5]

Archive location:

```
s3://noaa-gfs-bdp-pds/gfs.YYYYMMDD/CC/atmos/
```

where `CC` = forecast cycle (00, 06, 12, 18).[4] [5]

NOT VERIFIED — DO NOT IMPLEMENT YET: Exact file names for 2023/2024 monsoon season. You must verify the directory structure by listing:

```
aws s3 ls --no-sign-request s3://noaa-gfs-bdp-pds/gfs.20230701/00/atmos/
```

2.2 Observation Data (IMD)

Property	Value	Source
Dataset	IMD 0.25° Daily Gridded Rainfall	[6] [7]
Period	1901–2024 (verified); 2025 status NOT VERIFIED	[6]
Spatial grid	135 × 129 points	[6] [8]
Latitude	6.5°N to 38.5°N (0.25° spacing)	[6] [7]
Longitude	66.5°E to 100.0°E (0.25° spacing)	[6] [7]
Units	mm (daily accumulation)	[6]
File format	NetCDF (`.nc`)	[6] [8]
Variable name	`RAINFALL`	[8]
Missing value	`-999`	[8]
Access	https://www.imdpune.gov.in/cmpg/Griddata/Rainfall_25_NetCDF.html	[6] [7]

File naming convention:

```
RF25_ind<YYYY>_rfp25.nc
```

Example: `RF25_ind2023_rfp25.nc`. [9]

Citation requirement: Pai et al. (2014), MAUSAM, 65, 1, pp1-18. [6]

3. Verified Data Access

3.1 GFS Forecast Access

Primary method: AWS S3 (no authentication required)

```
aws s3 ls --no-sign-request s3://noaa-gfs-bdp-pds/gfs.20230701/00/atmos/
```

Expected files:

```
gfs.t00z.pgrb2.0p25.f000
gfs.t00z.pgrb2.0p25.f003
gfs.t00z.pgrb2.0p25.f006
...
gfs.t00z.pgrb2.0p25.f384
```

.

Alternative access:

- NCEI GDEX: https://gdex.ucar.edu/datasets/d084001/ [10]
- FTP (NOT RECOMMENDED for students due to volume): https://ftpprd.ncep.noaa.gov/data/nccf/com [11]

3.2 IMD Observation Access

Official portal: https://www.imdpune.gov.in/cmpg/Griddata/Rainfall_25_NetCDF.html. [7] [6]

Download procedure:

1. Select year (e.g., 2023)
2. Click download link (generates `RF25_ind2023_rfp25.nc`)
3. No registration required for historical years. [6]

NOT VERIFIED — DO NOT IMPLEMENT YET: Whether 2024/2025 files are currently available. Check the dropdown on the official page.

4. Exact Forecast Variable (GFS Precipitation)

Property	Value	Source
Parameter	APCP (Total Precipitation)	[12] [13] [14]
GRIB name	`APCP:surface`	[12] [15]
Level	Surface	[13] [14]
Units	kg/m² (equivalent to mm)	[13]
Step type	Accumulation (`accum`)	[16] [15]
Accumulation period	3-hourly for most forecast hours	[14] [17]

4.1 Accumulation Semantics (CRITICAL)

Verified from multiple sources:

For forecast hours **evenly divisible by 6** (FH000, FH006, FH012, FH018, FH024, …):

- Accumulation period = **T-6h to T** (6-hour accumulation). [14] [17]

For forecast hours **divisible by 3 but not 6** (FH003, FH009, FH015, FH021, …):

- Accumulation period = **T-3h to T** (3-hour accumulation). [17] [14]

Example calculation for 24-hour rainfall:

Initialization: `2023-07-01 00 UTC`

Forecast Hour	Valid Time	Accumulation Period	File
FH003	2023-07-01 03 UTC	00–03 UTC	`gfs.t00z.pgrb2.0p25.f003`
FH006	2023-07-01 06 UTC	00–06 UTC	`gfs.t00z.pgrb2.0p25.f006`
FH009	2023-07-01 09 UTC	06–09 UTC	`gfs.t00z.pgrb2.0p25.f009`
FH012	2023-07-01 12 UTC	06–12 UTC	`gfs.t00z.pgrb2.0p25.f012`
FH015	2023-07-01 15 UTC	12–15 UTC	`gfs.t00z.pgrb2.0p25.f015`
FH018	2023-07-01 18 UTC	12–18 UTC	`gfs.t00z.pgrb2.0p25.f018`
FH021	2023-07-01 21 UTC	18–21 UTC	`gfs.t00z.pgrb2.0p25.f021`
FH024	2023-07-01 24 UTC	18–24 UTC	`gfs.t00z.pgrb2.0p25.f024`

Correct 24-hour total (00–24 UTC):

```
APCP_FH024 (18–24h) + APCP_FH018 (12–18h) + APCP_FH012 (06–12h) + APCP_FH006 (00–06h)
```

NOT: Do not sum all 3-hourly files (FH003, FH006, FH009, …) — this would double-count overlapping periods. [12] [15]

Target rainfall period: 00–24 UTC on valid date.

Observed rainfall period: IMD daily rainfall is defined as 24-hour accumulation ending at 0830 IST (0300 UTC) on the calendar date. [18]

IMPORTANT MISMATCH: GFS 00–24 UTC ≠ IMD 0300–0300 UTC. You must either:

- **Option A (recommended):** Shift GFS forecast by +3 hours (use FH003–FH027)
- **Option B:** Accept 3-hour offset and document as limitation

5. Exact Observation Variable (IMD Rainfall)

Property	Value	Source
NetCDF variable	`RAINFALL`	[8]
Dimensions	`(TIME, LATITUDE, LONGITUDE)`	[8]
Units	`mm`	[6] [8]
Missing value	`-999`	[8]
Fill value	`-999`	[8]
Temporal definition	Daily (0300 UTC to 0300 UTC next day)	[18]

Grid orientation:

- First grid point: 6.5°N, 66.5°E [7] [6]
- Last grid point: 38.5°N, 100.0°E [6]
- Spacing: 0.25° in both directions [6]

6. Temporal Matching (Forecast + Observation Alignment)

6.1 Time Definition

Term	Definition
`forecast_init`	GFS initialization time (e.g., `2023-07-01 00:00 UTC`)
`lead_time`	Forecast hour (e.g., `24` for FH024)
`valid_time`	`forecast_init + lead_time` (e.g., `2023-07-02 00:00 UTC`)
`valid_date`	Calendar date of `valid_time` (e.g., `2023-07-02`)
`obs_date`	IMD observation date (e.g., `2023-07-02`)

6.2 Alignment Algorithm

Step 1: For each IMD observation date `D` (e.g., `2023-07-02`):

- IMD period: `D-1 03:00 UTC` to `D 03:00 UTC` [18]

Step 2: Select GFS forecast:

- Initialization: `D-1 00:00 UTC`
- Lead times: FH003, FH006, FH009, FH012, FH015, FH018, FH021, FH024, FH027
- Valid period: `D-1 00:00 UTC` to `D 03:00 UTC`

Step 3: Calculate 24-hour GFS rainfall for period `D-1 03:00 UTC` to `D 03:00 UTC`:

```
GFS_rain_24h = APCP_FH027(24-27h) + APCP_FH024(18-24h)
             - APCP_FH003(00-03h) + APCP_FH018(12-18h)
             + APCP_FH012(06-12h) + APCP_FH006(00-06h)
```

Simplified approach (recommended for first experiment):

- Use GFS initialization `D-1 00:00 UTC`
- Sum non-overlapping 6-hour accumulations: FH006 + FH012 + FH018 + FH024
- This gives 00–24 UTC rainfall (3-hour offset from IMD)
- Document this as a known limitation

6.3 UTC Handling

- All GFS times are in UTC. [1]
- IMD observation times are in IST but stored as UTC-equivalent (0300 UTC = 0830 IST). [18]
- Python: Use `pandas.Timestamp` with `tz='UTC'` for all calculations.
- Leap years: IMD NetCDF files contain 365 or 366 time records depending on year. [6]

7. Spatial Matching

7.1 Grid Comparison

Property	GFS 0.25°	IMD 0.25°
Resolution	0.25°	0.25°
Latitude range	Global (90°S–90°N)	6.5°N–38.5°N [6]
Longitude range	Global (0–360°E)	66.5°E–100.0°E [6]
Grid points	1440 × 721	135 × 129 [6]
Origin	NOT VERIFIED — check GRIB metadata	6.5°N, 66.5°E [6]

7.2 Recommended Approach

Option 1: Interpolate GFS → IMD grid (RECOMMENDED)

Rationale:

- IMD grid is the verification target (operational standard)
- Preserves observation resolution
- Scientifically defensible for rainfall verification [19]
- Easier to aggregate to districts later

Method:

1. Extract GFS APCP on native grid
2. Use bilinear interpolation to IMD grid coordinates
3. Mask to India domain (lat 6.5–38.5, lon 66.5–100)

Libraries: `xarray`, `cfgrib`, `scipy.interpolate`

Option 2: Aggregate IMD → GFS grid (NOT RECOMMENDED)

- Loses IMD resolution advantage
- Not scientifically defensible for verification

Option 3: Common sub-grid (NOT RECOMMENDED)

- Wasteful of data
- Complicates district aggregation

8. Training Dataset Schema

8.1 Verified Available Features (GFS)

From GFS pgrb2.0p25 inventory: [20]

Variable	GRIB Parameter	Level	Verified
`nwp_rain`	APCP	surface	✅ YES [12] [14]
`temperature_2m`	TMP	2m above ground	✅ YES [20]
`relative_humidity_2m`	RH	2m above ground	✅ YES [20]
`wind_u_10m`	UGRD	10m above ground	✅ YES [20]
`wind_v_10m`	VGRD	10m above ground	✅ YES [20]
`pressure_msl`	PRMSL	mean sea level	✅ YES [20]
`cape`	CAPE	surface	✅ YES [20]
`total_cloud_cover`	TCDC	entire atmosphere	✅ YES [20]

8.2 NOT VERIFIED — DO NOT IMPLEMENT YET

Variable	Status
`geopotential_height`	NOT VERIFIED at surface level in pgrb2.0p25
`vertical_velocity`	NOT VERIFIED at surface level
`precipitable_water`	PWAT exists [20] but NOT VERIFIED for this application

8.3 Final Minimal Feature Set (Phase 1)

```
{
    'forecast_init': 'datetime64[ns]',  # GFS initialization time
    'valid_date': 'datetime64[ns]',     # IMD observation date
    'lead_time': 'int16',               # Forecast hour (24)
    'latitude': 'float32',              # IMD grid latitude
    'longitude': 'float32',             # IMD grid longitude
    'nwp_rain': 'float32',              # GFS 24h APCP (mm)
    'temp_2m': 'float32',               # GFS 2m temperature (K)
    'rh_2m': 'float32',                 # GFS 2m relative humidity (%)
    'wind_u_10m': 'float32',            # GFS 10m u-wind (m/s)
    'wind_v_10m': 'float32',            # GFS 10m v-wind (m/s)
    'cape': 'float32',                  # GFS CAPE (J/kg)
    'observed_rain': 'float32',         # IMD rainfall (mm)
    'regime': 'category',               # Active/Break/Depression/Other
}
```

9. Train/Validation/Test Split

Recommended split (based on monsoon season):

Period	Years	Purpose
Training	2020–2022	Model fitting
Validation	2023	Hyperparameter tuning
Test	2024	Final evaluation (NOT used for training)

Rationale:

- 3 years training = ~366 days × 135 × 129 grid points ≈ 6 million samples
- Sufficient for XGBoost on CPU
- Temporal split prevents data leakage
- 2024 test set is independent (future data)

NOT VERIFIED — DO NOT IMPLEMENT YET: Whether 2024 IMD data is currently available. If not, use 2019–2021 train, 2022 validation, 2023 test.

10. Baseline B1: Raw NWP

Input: `nwp_rain` (GFS APCP 24h)
Target: `observed_rain` (IMD rainfall)
No ML — direct comparison

10.1 Metrics to Calculate

Metric	Formula	Thresholds	Source
RMSE	`sqrt(mean((pred - obs)^2))`	All rain	Standard
MAE	`mean(abs(pred - obs))`	All rain	Standard
Bias	`mean(pred) / mean(obs)`	All rain	Standard
CSI	`hits / (hits + misses + false_alarms)`	1, 10, 50, 100 mm	[19]
POD	`hits / (hits + misses)`	1, 10, 50, 100 mm	[19]
FAR	`false_alarms / (hits + false_alarms)`	1, 10, 50, 100 mm	[19]
ETS	`(hits - hits_random) / (hits + misses + false_alarms - hits_random)`	1, 10, 50, 100 mm	Standard
FSS	See Section 17	1, 10, 50 mm	[19]

IMD Rainfall Thresholds (verified):

- Light: 2.5–15.5 mm [18]
- Moderate: 15.6–64.4 mm [18]
- Heavy: 64.5–115.5 mm [21] [18]
- Very Heavy: 115.6–204.4 mm [21] [18]
- Extremely Heavy: ≥204.5 mm [21] [18]

Operational thresholds for verification:

- 1 mm (rain/no-rain)
- 10 mm (moderate rain)
- 50 mm (heavy rain warning) [22]
- 100 mm (very heavy rain warning) [22]

11. Bias Correction B2

11.1 Recommended Method: Quantile Mapping

Rationale:

- Handles non-Gaussian rainfall distribution
- Corrects both mean and variance
- Scientifically defensible for precipitation [19]

Formula:

```
corrected_rain = F_obs^{-1}(F_nwp(nwp_rain))
```

where:

- `F_nwp` = CDF of NWP rainfall (training period)
- `F_obs^{-1}` = inverse CDF of observed rainfall (training period)

Implementation:

1. Calculate empirical CDFs from training data (2020–2022)
2. For each NWP rainfall value, find its percentile in `F_nwp`
3. Map to observed rainfall at same percentile using `F_obs^{-1}`
4. Apply to validation/test data

Libraries: `scipy.stats`, custom implementation

11.2 Alternative: Mean Bias Correction (Simpler)

Formula:

```
corrected_rain = nwp_rain * (mean_obs_train / mean_nwp_train)
```

Use if: Quantile mapping proves too complex for initial implementation.

12. Generic ML B3

12.1 Model Selection: XGBoost

Justification:

- Handles tabular data efficiently
- Built-in handling of missing values
- Non-linear relationships (rainfall skewness)
- CPU training (no GPU required)
- Explainability via feature importance
- Proven in precipitation post-processing [19]

NOT RECOMMENDED:

- Deep learning (overkill for tabular data)
- Transformers (no sequence modeling needed)
- Random Forest (XGBoost typically outperforms)

12.2 Training Configuration

```
xgb.XGBRegressor(
    n_estimators=500,
    max_depth=6,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    objective='reg:squarederror',
    tree_method='hist',  # CPU-efficient
    n_jobs=-1
)
```

Target: `observed_rain`
Features: All verified GFS variables (Section 8.3)

13. Regime Classifier

13.1 Regime Definitions (Verified from Scientific Literature)

Regime	Definition	Source
Active	Normalized rainfall anomaly > +1.0 for ≥3 consecutive days over core monsoon zone	[23] [24] [25]
Break	Normalized rainfall anomaly < -1.0 for ≥3 consecutive days over core monsoon zone	[23] [24] [25]
Depression/Low	LPS track within 3° of grid point with wind speed <17 kt (low) or 17–27 kt (depression)	[26] [27] [28]
Other/Normal	Does not meet above criteria	Derived

Core monsoon zone: NOT VERIFIED — typically 21–27°N, 73–82°E based on, but confirm from IMD documentation. [29] [30]

13.2 Recommended Approach: Rule-Based Classifier

Rationale:

- Scientifically defensible (published criteria)
- No training data required
- Transparent and explainable
- Avoids ML complexity for first version

Algorithm:

1. Calculate daily rainfall anomaly for core monsoon zone
2. Normalize by climatology (1901–2020 mean)
3. Label days as Active/Break based on ±1.0 threshold
4. Overlay LPS track data (if available) for Depression label

NOT VERIFIED — DO NOT IMPLEMENT YET:

- Historical LPS track database availability
- Exact core monsoon zone coordinates from IMD

Fallback: Use only Active/Break regimes initially (no Depression label).

14. Regime-Aware ML B4

14.1 Recommended Architecture: Single Model with Regime Feature

Architecture B (from your options):

```
NWP variables + regime_label
        ↓
     XGBoost
```

Rationale:

- Avoids sparse-regime problems (few Depression days)
- Easy to implement (add `regime` as categorical feature)
- Directly tests PS 26080 hypothesis
- Computationally feasible

NOT RECOMMENDED:

- Separate models per regime (sparse data for Depression)
- Mixture of experts (unnecessary complexity)

14.2 Implementation

```
# Add regime as one-hot encoded feature
features = ['nwp_rain', 'temp_2m', 'rh_2m', ..., 'regime_active', 'regime_break', 're
```

15. Heavy Rainfall Probability

15.1 Recommended Approach: Quantile Regression

Method:

- Train XGBoost with `objective='reg:quantile'`
- Predict 90th, 95th, 99th percentiles
- Probability = fraction of quantiles exceeding threshold

Alternative: Binary Classifier

- Train XGBClassifier for `observed_rain > threshold`
- Output = calibrated probability

Recommended threshold: 64.5 mm (IMD Heavy Rain) [21] [18]

16. Evaluation Metrics

16.1 Required Metrics (per PS 26080)

Metric	Purpose	Must Improve?
RMSE	Overall accuracy	YES
CSI	Critical Success Index	YES
POD	Probability of Detection	YES
FAR	False Alarm Ratio	NO (but should not worsen significantly)
ETS	Equitable Threat Score	YES
FSS	Spatial skill	YES

16.2 Claiming Improvement

Valid claim: "Our model improved the rainfall forecast" requires:

- Lower RMSE than B1 (raw NWP)
- Higher CSI at ≥2 thresholds (e.g., 10 mm, 50 mm)
- Higher POD without large FAR increase
- Higher ETS

NOT sufficient: RMSE improvement alone (may hide heavy-rain degradation).

17. Spatial Evaluation (FSS)

17.1 Fractions Skill Score

Formula:

```
FSS = 1 - (1/n) * Σ((p_obs - p_pred)^2) / (1/n * (Σp_obs^2 + Σp_pred^2))
```

where `p` = fraction of grid points exceeding threshold in neighborhood.

Recommended neighborhood sizes:

- 3×3 grid (0.75° × 0.75°)
- 5×5 grid (1.25° × 1.25°)

- 7×7 grid (1.75° × 1.75°)

Rationale: Accounts for spatial displacement errors common in NWP. [19]

18. District Product

18.1 Recommended Approach: Grid → District Aggregation

Option A (RECOMMENDED):

1. Predict on IMD grid (0.25°)
2. Aggregate to districts using area-weighted mean

Rationale:

- Preserves meteorological grid information
- Scientifically defensible
- Easier to verify

District boundaries:

- Source: India Data Portal or Survey of India
- Format: GeoJSON/Shapefile
- NOT VERIFIED — confirm data availability

19. Software Architecture

```
PS26080/
├── data/
│   ├── raw/
│   │   ├── gfs/          # GRIB2 files
│   │   └── imd/          # NetCDF files
│   ├── interim/          # Processed GRIB → NetCDF
│   └── processed/        # Final training CSV/Parquet
│
├── src/
│   ├── data/
│   │   ├── download_gfs.py
│   │   └── download_imd.py
│   ├── preprocessing/
│   │   ├── read_gfs.py
│   │   ├── read_imd.py
│   │   ├── align_time.py
│   │   └── align_grid.py
│   ├── regimes/
│   │   └── classify_regime.py
│   ├── models/
│   │   ├── baseline_raw.py
│   │   ├── bias_correction.py
│   │   ├── xgboost_model.py
```

```
│   │   └── regime_aware.py
│   ├── evaluation/
│   │   ├── metrics.py
│   │   └── spatial_eval.py
│   └── visualization/
│       ├── plot_maps.py
│       └── dashboard.py
│
├── configs/
│   ├── data_config.yaml
│   └── model_config.yaml
│
├── notebooks/
│   ├── 01_data_exploration.ipynb
│   └── 02_model_comparison.ipynb
│
├── models/                 # Saved XGBoost models
├── outputs/                # Predictions, metrics
├── reports/                # Figures, tables
├── tests/                  # Unit tests
├── requirements.txt
└── README.md
```

20. Python Libraries

Library	Purpose	Required?
`xarray`	GRIB2/NetCDF I/O	YES
`cfgrib`	GRIB2 engine (via xarray)	YES
`pandas`	Tabular data	YES
`numpy`	Numerical ops	YES
`scikit-learn`	Metrics, preprocessing	YES
`xgboost`	ML model	YES
`geopandas`	District boundaries	YES
`rasterio`	Spatial operations	YES
`matplotlib`	Plotting	YES
`cartopy`	Maps	OPTIONAL
`netCDF4`	NetCDF backend	YES (via xarray)
`scipy`	Interpolation	YES

21. Development Milestones

Milestone	Description	Validation Check
M0	Verify GFS data access	Can list `s3://noaa-gfs-bdp-pds/gfs.20230701/00/atmos/`
M1	Download one GFS forecast	File `gfs.t00z.pgrb2.0p25.f024` exists
M2	Read GRIB2 with xarray	APCP variable loads without error
M3	Download IMD NetCDF	File `RF25_ind2023_rfp25.nc` exists
M4	Read IMD NetCDF	`RAINFALL` variable loads, shape=(365,129,135)
M5	Match one day	Single (forecast, obs) pair aligned
M6	Create 7-day sample	7 days × grid points in DataFrame
M7	Create one-month dataset	July 2023 complete
M8	Run raw-NWP verification	RMSE, CSI calculated
M9	Bias correction	Quantile mapping applied
M10	XGBoost model	Trained on 2020–2022
M11	Regime labels	Active/Break days classified
M12	Regime-aware XGBoost	Model with regime feature
M13	Heavy rain probability	P(rain>64.5mm) output
M14	District aggregation	Grid → district map
M15	Dashboard	Streamlit app (optional)

DO NOT proceed to next milestone until current one passes validation.

22. Validation Checks

22.1 Data Validation

- GFS APCP units = kg/m^2 (verify via GRIB metadata)
- IMD RAINFALL units = mm (verify via NetCDF attributes)
- No NaN in IMD data (replace -999 with NaN)
- Spatial alignment verified (plot GFS vs IMD for one day)

22.2 Model Validation

- Training data leakage check (no test data in training)
- Regime labels calculated from observations only (not forecasts)
- Bias correction parameters from training set only

23. Stop Conditions

Condition	Fallback
SC1: GFS historical data inaccessible	Use Level 3 fallback (small domain, recent data)
SC2: APCP accumulation unclear	Use only FH006+FH012+FH018+FH024 (6-hourly)
SC3: IMD data unavailable for 2023–2024	Use 2019–2022 instead
SC4: Regime labels unreliable	Skip regime-aware model (B3 only)
SC5: No improvement over raw NWP	Document as scientific result (negative finding)
SC6: Heavy rain detection worsens	Report trade-off; do not claim overall improvement

24. Fallback Plan

Level	Data	Scope	Claims Allowed
L1 (Ideal)	NCMRWF NCUM + IMD	Full India, all regimes	Full PS 26080
L2 (Reproducible)	GFS + IMD	Full India, Active/Break	"Improved GFS rainfall forecast"
L3 (Reduced)	GFS + IMD	Core monsoon zone only	"Proof of concept"
L4 (Algorithm only)	Small sample	1 month, single regime	"Methodology validated"

This project targets L2.

25. Known Limitations

1. **3-hour time offset:** GFS 00–24 UTC ≠ IMD 0300–0300 UTC
2. **No depression regime (initially):** LPS track data NOT VERIFIED
3. **GFS ≠ NCUM:** Model differences not accounted for
4. **No real-time capability:** Historical only
5. **No ensemble:** Single deterministic forecast

26. Claims Allowed

✅ **Allowed:**

- "ML post-processing improved GFS 24-hour rainfall forecast over India during monsoon 2020–2024"
- "Regime-aware model outperformed generic ML for heavy rainfall"
- "Quantile mapping reduced RMSE by X%"

❌ **NOT Allowed:**

- "Improved NCMRWF NCUM forecast" (did not use NCUM)

- "Operational forecasting system" (not operational)
- "Real-time predictions" (historical only)
- "All six regimes" (only Active/Break implemented)

27. What NOT to Build (Initially)

- ✗ Custom NWP model
- ✗ Deep learning / Transformers
- ✗ Real-time deployment
- ✗ 10-day forecasting
- ✗ All six regimes
- ✗ Complex microservices
- ✗ Mobile application
- ✗ IoT hardware
- ✗ Live forecast dashboard

Focus: Scientific validation first, engineering later.

28. Next Steps

1. **Verify GFS access:**
   ```
   aws s3 ls --no-sign-request s3://noaa-gfs-bdp-pds/gfs.20230701/00/atmos/ | head
   ```
2. **Verify IMD 2023 availability:**
   Visit https://www.imdpune.gov.in/cmpg/Griddata/Rainfall_25_NetCDF.html
3. **Download one GFS file + one IMD file**
4. **Attempt M5 (match one day)**
5. **Report back:** Any blockers at M0–M5?

This specification is designed to be executable by a student team using ordinary laptops. All components are verified from official sources or explicitly marked as NOT VERIFIED.

⁂

1. https://www.ncei.noaa.gov/products/weather-climate-models/global-forecast
2. https://vlab.noaa.gov/web/gfs/products
3. https://www.nco.ncep.noaa.gov/pmb/products/gfs/
4. https://registry.opendata.aws/noaa-gfs-bdp-pds/
5. https://herbie.readthedocs.io/en/stable/gallery/noaa_models/gfs.html
6. https://www.imdpune.gov.in/cmpg/Griddata/Rainfall_25_NetCDF.html

7. https://imdpune.gov.in/cmpg/Griddata/Rainfall_25_NetCDF.html
8. https://ankitdeshmukh.com/post/2024-10-03-working-with-netcdf/
9. https://github.com/Veethahavya/IMD-gridded_rainfall_fetch_vis
10. https://gdex.ucar.edu/datasets/d084001/
11. https://herbie.readthedocs.io/en/2024.8.0/gallery/noaa_models/gfs.html
12. https://www.cpc.ncep.noaa.gov/products/wesley/wgrib2/unmerge_fcst.html
13. https://www.ftp.cpc.ncep.noaa.gov/wd51we/wgrib2/intro_grib2.pdf
14. https://www.villasmunta.it/opengrads/variabili_precipitazioni.htm
15. https://wgrib2-docs.readthedocs.io/en/latest/options/output/unmerge_fcst/
16. https://ufs2arco.readthedocs.io/en/latest/gfs_archive.html
17. http://gradsusr.org/pipermail/gradsusr/2012-May/032485.html
18. https://mausam.imd.gov.in/imd_latest/contents/pdf/forecasting_sop.pdf
19. https://mausamjournal.imd.gov.in/index.php/MAUSAM/article/download/548/474/1848
20. https://www.nco.ncep.noaa.gov/pmb/products/gfs/gfs.t00z.pgrb2.0p25.f000.shtml
21. https://nwp.imd.gov.in/Report_2_2025.pdf
22. https://nwp.imd.gov.in/mme/fdp-bob/Souvenir_final_17sep.pdf
23. https://www.imdpune.gov.in/seasons_previous/swmonsoon/swmonsoon24/active_break.html
24. https://www.ias.ac.in/describe/article/jess/119/03/0229-0247
25. https://drs.nio.res.in/drs/bitstream/handle/2264/4986/Clim_Dyn_46_3921a.pdf?sequence=1&isAllowed=y
26. https://link.springer.com/article/10.1007/s00382-019-04744-x
27. https://journals.ametsoc.org/view/journals/clim/23/16/2010jcli2953.1.xml
28. https://mausamjournal.imd.gov.in/index.php/MAUSAM/article/download/1102/937/4030
29. https://journals.ametsoc.org/view/journals/atsc/57/9/1520-0469_2000_057_1354_dobiti_2.0.co_2.xml
30. https://www.tropmet.res.in/~lip/Publication/RR-pdf/RR-112.pdf
31. https://www.imdpune.gov.in/hindi/cmpg/Rainfall_25_NetCDF_Hindi.html
32. https://imdpune.gov.in/cmpg/Griddata/Rainfall_25_Bin.html
33. https://imdpune.gov.in/cmpg/Realtimedata/gpm/Rain_Download.html
34. https://www.imdpune.gov.in/cmpg/Griddata/Rainfall25Merged.html
35. https://iridl.ldeo.columbia.edu/SOURCES/.IMD/.RF0p25/.gridded/.daily/index.html?Set-Language=en
36. https://iridl.ldeo.columbia.edu/SOURCES/.IMD/overview.html
37. https://iridl.ldeo.columbia.edu/SOURCES/.IMD/.RF0p25/.gridded/.daily/.v1901-2015/.rf/index.html?Set-Language=en
38. https://rcc.imdpune.gov.in/download.php
39. https://indiadataatlas.deccanspatial.com/chapters/ch03-environment/imd-weather/
40. https://waterinag.github.io/Data4WA/datasets/pcp_imd/
41. https://imdlib.readthedocs.io/en/latest/Usage.html
42. https://journals.ametsoc.org/view/journals/clim/36/1/JCLI-D-21-1003.1.xml
43. https://journals.ametsoc.org/view/journals/clim/20/1/jcli3981.1.xml

44. https://journals.ametsoc.org/view/journals/clim/29/21/jcli-d-16-0028.1.xml
45. https://drs.nio.res.in/drs/bitstream/handle/2264/187/Geophys_Res_Lett_31_L18201.pdf?sequence=3&isAllowed=y
46. https://www.ias.ac.in/public/Volumes/jess/112/04/0529-0558.pdf
47. https://egusphere.copernicus.org/preprints/2025/egusphere-2025-4585/egusphere-2025-4585.pdf
48. https://www.ias.ac.in/describe/article/jess/112/04/0529-0558
49. https://www.cpc.ncep.noaa.gov/products/wesley/wgrib2/ncep_norm.html
50. https://www.ncei.noaa.gov/access/search/
51. https://waterinag.github.io/eqipa-docs/datasets/pcp_imd/
52. https://centos.imr.no/cran/web/packages/imdR/imdR.pdf
53. https://herbie.readthedocs.io/en/2025.2.1/gallery/noaa_models/gfs.html
54. https://noaa-gfs-bdp-pds.s3.amazonaws.com/
55. https://api.imd.gov.in/public/api_reference.html
56. https://docs.usdata.dev/providers/noaa-services/
57. http://gradsusr.org/pipermail/gradsusr/2019-June/042407.html
58. https://link.springer.com/chapter/10.1007/978-981-15-4327-2_7
59. https://journals.ametsoc.org/view/journals/atsc/75/6/jas-d-17-0309.1.xml
60. https://www.nature.com/articles/s41612-023-00436-w
61. https://journals.ametsoc.org/view/journals/clim/23/4/2009jcli2918.1.xml
62. https://tellusjournal.org/articles/10.3402/tellusa.v67.27039
63. http://repository.ias.ac.in/23797/1/324.pdf
64. https://zenodo.org/records/11195106
65. https://www.ncei.noaa.gov/thredds/catalog/model-gfs-g3-anl-files-old/catalog.html
66. https://imdpune.gov.in/lrfindex.php
67. https://dsp.imdpune.gov.in/home_gridded_climatology.php
68. https://dsp.imdpune.gov.in/home_freedataaccess.php
69. https://www.scribd.com/document/874592503/s41598-024-75320-5
70. https://pmc.ncbi.nlm.nih.gov/articles/PMC11589684/
71. https://www.nco.ncep.noaa.gov/pmb/products/gfs/gfs.t00z.pgrb2.0p25.f003.shtml
72. https://wgrib2-docs.readthedocs.io/en/latest/options/output/ncep_norm.html